Integration Test Report

Deliverable 3, Campus Ride-Pooling and Split-Fare System

Prepared by Mainak Sarkar Jira CS455-110, CS455-111, CS455-112, CS455-114 Run of 2026-11-02

Prithviraj Ghosh (230794) Mainak Sarkar (230619) Khushi Ranawat (230562) Mahir Jain (230618) Chaitanya Nitawe (230315)

Result. 68 of 68 integration tests passed in 10 files (9 of them regression tests for defects this suite found), against a real PostgreSQL 16 database, locally and in GitHub Actions. Together with the 41 adversarial cases (separate report) the run had 109/109 passing in 32.8 s of test time. Two defects were found and fixed with regression tests (Section 6).

1. Purpose and scope

The unit suite (about 1,500 tests, reported separately) checks each module in isolation, with the database and the session mocked. It cannot see a query that PostgreSQL rejects, a constraint or trigger that refuses a write, or two modules disagreeing about the rows they share. This suite closes that gap: every test calls the *real* API route handlers, which run their real SQL against a real database, and checks both the HTTP answer and the rows written.

In scope: the core transactional workflow of the D2 demonstration (offer, search, request, decide, lock, pickup, trip, completion, rating, complaint) and the safety and operations flows around it (SOS and incident handling, warnings, suspensions, payment reminders). Out of scope, and covered elsewhere: rendered pages and the browser (system tests), the 50-way last-seat experiment (concurrency report), access control and IDOR (security report), latency (performance report).

2. Test harness

2.1 What runs for real, and what is replaced

Real	Route handlers, session lookup (`getCurrentUser`), the same-origin check and database rate limit (`guard`), the admin role check, every SQL query, constraint, trigger and transaction, the Gemini fallback path.
Replaced	`next/headers` (`cookies()`, `headers()`) and `next/server` (`after()`): Next.js answers them only inside its own request handling. Small shims read the current test request from `AsyncLocalStorage`; work passed to `after()` runs right after the handler answers, as Next.js runs it after the response.
Switched off	Outgoing email and Gemini (empty keys), so no test calls an external service; the app's documented fallbacks run instead.

Routes are called in-process with `callRoute(handler, {method, body, params, as, origin, ip})`, which builds the `Request`, sets the session cookie of the caller, a same-origin or cross-site `Origin`, and a fresh client address per call (so per-address rate limits are only exercised on purpose). In-process calls need no production build, run the whole suite in under a minute, and give full stack traces; the deployed stack end to end is the system tests' job.

2.2 Test data and time

- **Accounts and sessions** are written with SQL (`test/seed/accounts.ts`), because registering through the app sends a real verification email. Only a token's SHA-256 is stored, exactly as the app stores it. The same seed is shared with the Playwright system tests.
- **Everything else goes through the API**: rides are offered, seats requested and accepted through the real routes, so the rows are those the app itself writes.
- **Time** is the one thing simulated: `moveDeparture(ride, minutes)` moves a ride's departure, and the departure and pickup times its riders agreed to, by the same amount, as if the clock had moved on (the move `setup.md` documents for trying a trip).

2.3 Isolation and safety

Each run drops and recreates the test database and applies `db/schema.sql`, the file production migrations apply, so the schema is also checked on an empty database every run. Before every test all tables are truncated and the rows the schema itself inserts (campus places, hubs, vehicle types, tags) are put back, so every test starts from a freshly migrated database.

Because that is destructive, the harness fails closed (`test/integration/test-database.ts`): it touches a database only if (1) its name ends in `_test`, so the application database is never a candidate; (2) it is on this machine (`localhost`, `127.0.0.1` or `::1`), never a hosted server; (3) `ALLOW_TEST_DATABASE_RESET` names exactly that database, an explicit opt-in; and (4) the harness created it. Global setup writes a marker comment on the database it creates and refuses to drop an existing one without it; each test file checks the marker again before its first reset. These checks were added after review (PR #83); unit tests cover each refusal.

3. Environment and how to rerun

Runtime	Node.js 24, Next.js 16.3, Vitest 5.0.3, node-postgres 8
Database	PostgreSQL 16: Docker Compose locally (`127.0.0.1:5433`), a service container in CI
CI	Job *Integration tests (PostgreSQL)* in `.github/workflows/ci.yml`, on every PR and push to master; latest run on the stacked PRs: run 40850333045 (pass)

```
docker compose up -d db                                                  (from the repository root)
TEST_DATABASE_URL=postgres://crp:<password>@127.0.0.1:5433/crp_test      (in apps/web/.env.local)
ALLOW_TEST_DATABASE_RESET=crp_test                                       (same file)
npm run test:integration                                                 (from apps/web)
```

The tables in this report: `Deliverables/D3/test-run/README.md`.

4. Results

Table 1. Integration test files, from the recorded run (`test-run/integration-results.json`).

File (under `apps/web/test/integration/`)	Tests	Passed	Time (s)
`harness.int.test.ts`	6	6	1.4
`admin/complaints-and-enforcement.int.test.ts`	9	9	3.3
`bugs/idempotency-key-reuse.int.test.ts`	5	5	1.7
`bugs/timestamp-parameters.int.test.ts`	4	4	1.0
`rides/join-requests.int.test.ts`	9	9	2.6
`rides/manage-ride.int.test.ts`	5	5	1.7
`rides/offer-and-search.int.test.ts`	7	7	1.9
`rides/seats-and-lock.int.test.ts`	7	7	2.3
`trip/sos.int.test.ts`	7	7	2.6
`trip/trip-flow.int.test.ts`	9	9	3.0
Total	**68**	**68**	

Table 2. Code coverage of the integration run alone (%), from `test-run/integration-coverage-summary.json`. Pages and components are not reachable through API routes, so the whole-`src` figure is low by design; the unit suite carries the 80% gate (NFR-RD-17).

Area	Statements	Branches	Functions	Lines
API route handlers (`src/app/api`)	35.5	29.6	43.6	39.7
Server library (`src/lib`)	34.3	26.0	37.0	36.7
All of `src` (pages and components included)	22.8	17.3	19.4	24.1

5. Requirements covered

Each row names the file that exercises the requirement; every case is listed in the appendix.

Requirement	File	Checked
FR-RO-01 to 03, P-08: offer a ride	`rides/offer-and-search`	stored ride and owner seat; field errors; overlap and two-hour conflict
FR-RD-03, FR-RD-08.1: search	`rides/offer-and-search`	seats left, estimated share, hidden own, joined, locked, full and filtered rides
FR-RD-06, FR-RD-06.9: request a seat, idempotency	`rides/join-requests`, `bugs/idempotency-key-reuse`	snapshot, history, key; one request per retried key, also when first uses race; limit of three
FR-RO-05: owner decides	`rides/join-requests`	accept, reject, share notices; only the owner decides
FR-RD-06.10, FR-RD-07: withdraw and leave	`rides/join-requests`, `rides/seats-and-lock`	seat freed before lock; late leave keeps the locked share
FR-RO-07, P-06: lock	`rides/seats-and-lock`	shares fixed owner first (334/333/333), waiting requests expired, lock at time
FR-RO-04: edit and cancel	`rides/manage-ride`	confirmation for accepted riders, reconfirmation after a moved departure, cancel until the window ends
FR-RO-09, FR-RD-10: trip states	`trip/trip-flow`	lock before pickup, no skipping, check-in, arrivals, start trip, completion
FR-RO-10, P-19: no-show	`trip/trip-flow`	refused inside the grace period, recorded after; share spread over the rest
FR-RO-09.4: final shares	`trip/trip-flow`	written once, sum equals the fare
FR-RD-15, FR-RO-13.2: SOS	`trip/sos`	one alert per person; no trace for the pool; snapshot, location, false alarm
CS455-65, SYS-NFR-08: incident handling	`trip/sos`	acknowledge, close with a note, every attempt audited; students refused
FR-RD-13, SYS-FR-46: complaints	`admin/complaints-and-enforcement`	window from completion or own no-show; one per person; Gemini fallback recorded; admin review with version check
FR-RD-12, P-16: ratings	`admin/complaints-and-enforcement`	once per rater; never self or a no-show
OA-FR-21 to 28: warnings and suspensions	`admin/complaints-and-enforcement`	warning blocks until acknowledged; suspension blocks offering and joining until lifted; database trigger
FR-RD-11: overdue payment, reminders	`bugs/timestamp-parameters`	request blocked after 7 days due; 24 h then 72 h reminder, once each

6. Defects found

#	Defect	Found by	Fix and test
1	**Every seat request answered 500 on a real database**, and so did the payment reminder scheduler. `completed_at < $2 - interval '7 days'` with a JavaScript `Date`: node-postgres sends it untyped, PostgreSQL reads `$2 - interval` as interval minus interval, and no operator compares a timestamp with an interval. Introduced on 14 Oct with the settlement feature; invisible to the mocked unit tests. Fixed by casting the parameter.	This harness, first run	PR #84; `bugs/timestamp-parameters` (4 tests, all fail without the fix)

2	**An idempotency key sent again for another ride or pickup point replayed the first request**: the rider was told "already submitted" and nothing was created for the ride asked for, or a changed pickup was dropped. Now refused with 409 `idempotency_key_reused`.	Adversarial suite	PR #87; `bugs/idempotency-key-reuse` (5 tests; the 2 mismatch cases fail without the fix)

Both are tracked as Jira Bug issues with the `d3-defect` label and listed in the team's bug history.

7. Test reliability

A flaky reminder test, explained and fixed. The payment reminder test failed about once in a dozen full runs (1 in 18 in a dedicated loop), with "The original final share cannot be changed". The test seeded the final shares with `created_at = now()` on the *database's* clock, and the reminder then stamped `reminder_24_at` with the *test's* clock a millisecond or two later. The PostgreSQL container's clock runs about 1 ms ahead of the laptop's (measured: median 0.98 ms, at most 1.7 ms), and a JavaScript `Date` drops microseconds, so the reminder could look older than its share, which the share trigger refuses. It is not an application defect: the app writes final shares at completion, and reminders come at least 24 hours later. The seed now dates the shares from the ride's completion, as `src/lib/trip.ts` does. With the laptop's clock simulated 1 s behind the database, the old seed failed 3 runs of 3 and the new one passes 3 of 3; after the fix, 29 consecutive full runs all passed.

Simultaneous first uses of one idempotency key (a review question on PR #87). Claiming a key is atomic: `submitJoinRequest()` runs in one transaction that first locks the rider's `users` row (`FOR UPDATE`), so one rider's submissions run one after another and a later one sees the earlier key, and the key table's primary key refuses a second claim regardless. A regression test sends four first uses of one key at once, two for each of two rides, in five rounds: one request is created, the same ride's other send replays it, the other ride's sends get 409. With the row lock removed that test fails: the race reaches the primary key and answers 500.

8. Limitations

- **No browser and no Socket.IO.** Pages, the live chat socket and the in-process timers in `server.mjs` and `instrumentation.ts` are not exercised here; the Playwright system tests cover them on the Docker stack.
- **Gemini's success path is not called**, only its fallback: tests must not depend on an external service or spend its quota. The AI paths are covered by unit tests with recorded responses.
- **Time is simulated by moving data**, not the clock. The moved fields are the ones the app reads for each rule; a rule that read another time field would not see the move.
- **Two clocks.** Rows seeded with SQL take the database's time, while the app stamps the test's. Seeds therefore date rows as the app would (Section 7), and time-window cases keep margins of seconds, far above the measured millisecond offset.

A. Every integration case

File	Case (describe block: test)	Result
`harness`	integration harness: offers a ride through the real API and reads it back from the database	Pass
`harness`	integration harness: runs the real sign-in and origin checks	Pass
`harness`	integration harness: reads the role from the database for admin routes	Pass
`harness`	integration harness: moves a ride into its lock window for trip tests	Pass
`harness`	a clean database for every test: writes a ride	Pass
`harness`	a clean database for every test: no longer sees it, and still has the schema reference data	Pass
`complaints-and-enforcement`	complaints (FR-RD-13): opens on completion, files once per person, and falls back to manual review without Gemini (FR-RD-13.1, FR-RD-13.2, FR-RD-13.4, SYS-NFR-09)	Pass
`complaints-and-enforcement`	complaints (FR-RD-13): refuses before the trip, outsiders, oneself and a too-short description (FR-RD-13.1)	Pass
`complaints-and-enforcement`	complaints (FR-RD-13): lets a rider marked as a no-show dispute it before the ride is completed (FR-RD-10.3, FR-RD-13.1)	Pass
`complaints-and-enforcement`	complaints (FR-RD-13): is reviewed by an admin with a version check, and every step is audited (OA-FR-18, SYS-NFR-08)	Pass

File	Case (describe block: test)	Result
complaints-and-enforcement	ratings (FR-RD-12, FR-RO-12): opens on completion, once per rater, never for oneself (FR-RD-12.1, FR-RD-12.2, FR-RO-12.1)	Pass
complaints-and-enforcement	ratings (FR-RD-12, FR-RO-12): is closed before completion and never covers a no-show (FR-RD-12.2, FR-RO-12.2)	Pass
complaints-and-enforcement	warnings and suspensions (OA-FR-21 to OA-FR-28): blocks a warned rider until they acknowledge the warning (OA-FR-21, FR-RD-01.6)	Pass
complaints-and-enforcement	warnings and suspensions (OA-FR-21 to OA-FR-28): stops a suspended rider offering or joining rides until an admin lifts it (OA-FR-23, OA-FR-28, SYS-FR-24, FR-RD-01.5)	Pass
complaints-and-enforcement	warnings and suspensions (OA-FR-21 to OA-FR-28): allows enforcement by admins only, in the API and in the database (SYS-NFR-03)	Pass
idempotency-key-reuse	an idempotency key sent again with other details (FR-RD-06.9): is refused for another ride, and creates nothing there	Pass
idempotency-key-reuse	an idempotency key sent again with other details (FR-RD-06.9): is refused for another pickup point, keeping the first request as it was	Pass
idempotency-key-reuse	an idempotency key sent again with other details (FR-RD-06.9): is claimed once when its first uses for different rides arrive at the same moment	Pass
idempotency-key-reuse	an idempotency key sent again with other details (FR-RD-06.9): cannot be claimed twice even past the application, by the key table itself	Pass
idempotency-key-reuse	an idempotency key sent again with other details (FR-RD-06.9): still replays the same request sent again unchanged	Pass
timestamp-parameters	timestamp parameters compared with an interval: lets a rider with no unpaid shares request a seat	Pass
timestamp-parameters	timestamp parameters compared with an interval: still refuses a rider whose share has been due for more than seven days (FR-RD-11.5)	Pass
timestamp-parameters	timestamp parameters compared with an interval: allows a share that has been due for less than seven days (FR-RD-11.5)	Pass
timestamp-parameters	timestamp parameters compared with an interval: sends each payment reminder once, 24 hours and then 72 hours after completion (FR-RD-11.4)	Pass
join-requests	requesting a seat (FR-RD-06): records a pending request with its snapshot, history and idempotency key, and tells the owner (FR-RD-06.1, FR-RO-05.5)	Pass
join-requests	requesting a seat (FR-RD-06): answers a retried request with the same idempotency key once, without a second request (FR-RD-06.9)	Pass
join-requests	requesting a seat (FR-RD-06): refuses the owner, an unknown pickup point and a fourth waiting request (FR-RD-06.2)	Pass
join-requests	requesting a seat (FR-RD-06): shows the owner the waiting requests and the rider their own history (FR-RO-05.1, FR-RD-06.10)	Pass
join-requests	the owner deciding (FR-RO-05): accepting gives the rider a seat, updates everyone's share and records why (FR-RO-05.2, FR-RO-05.3, FR-RO-07.2)	Pass
join-requests	the owner deciding (FR-RO-05): rejecting leaves the seat free and tells the rider (FR-RO-05.4, FR-RD-06.4)	Pass
join-requests	the owner deciding (FR-RO-05): lets only the owner decide, and only the rider withdraw (NFR-RO-SEC-01)	Pass
join-requests	the rider changing their mind (FR-RD-06.10, FR-RD-07): withdraws a waiting request once, and the owner is told (FR-RD-07.1)	Pass
join-requests	the rider changing their mind (FR-RD-06.10, FR-RD-07): leaves before lock with nothing owed, freeing the seat for others (FR-RD-07.2)	Pass
manage-ride	cancelling a ride (FR-RO-04.4): cancels every request and seat claim and tells each rider they owe nothing (FR-RD-06.7, FR-RD-06.11)	Pass
manage-ride	cancelling a ride (FR-RO-04.4): refuses anyone but the owner, and a ride whose window is over (FR-RO-04.5, NFR-RO-SEC-01)	Pass
manage-ride	editing a ride (FR-RO-04.1): asks the owner to confirm a change that affects accepted riders, then tells them (FR-RO-04.2, FR-RO-04.3)	Pass
manage-ride	editing a ride (FR-RO-04.1): sends a waiting request back to the rider when the departure moves more than 10 minutes (FR-RD-06.5, P-09)	Pass
manage-ride	editing a ride (FR-RO-04.1): refuses edits that break the ride's rules	Pass
offer-and-search	offering a ride (FR-RO-03): stores the ride with the vehicle capacity and gives the owner the first seat (FR-RO-03.1, FR-RO-03.3)	Pass
offer-and-search	offering a ride (FR-RO-03): refuses an invalid offer field by field and stores nothing (FR-RO-03.2)	Pass
offer-and-search	offering a ride (FR-RO-03): refuses a ride that overlaps or departs within two hours of the owner's other ride (FR-RO-03.2)	Pass
offer-and-search	searching for a ride (FR-RD-03, FR-RD-08.1): shows an open ride with its seats and the share a new rider would pay (FR-RD-03.1, FR-RD-03.3)	Pass

File	Case (describe block: test)	Result
offer-and-search	searching for a ride (FR-RD-03, FR-RD-08.1): never shows the owner their own ride, nor a rider a ride they are on (FR-RD-03.2)	Pass
offer-and-search	searching for a ride (FR-RD-03, FR-RD-08.1): hides rides that are locked, full, or outside the filters (FR-RD-03.2)	Pass
offer-and-search	searching for a ride (FR-RD-03, FR-RD-08.1): keeps a ride the viewer has asked to join visible with its request state (FR-RD-03.2)	Pass
seats-and-lock	the last seat: refuses a request when the ride is full, and an accept that would overfill it (FR-RD-06.2, FR-RD-06.3, FR-RO-05.2)	Pass
seats-and-lock	the last seat: withdraws a rider's other request within two hours once they get a seat (FR-RD-06.8, P-08)	Pass
seats-and-lock	locking the ride (P-06): fixes every share when the owner locks early, owner first, and closes waiting requests (FR-RO-06.2, FR-RD-08.3, FR-RD-06.7)	Pass
seats-and-lock	locking the ride (P-06): lets only the owner lock (FR-RO-06.2, NFR-RO-SEC-01)	Pass
seats-and-lock	locking the ride (P-06): locks by itself an hour before departure, on the next request (FR-RO-06.3)	Pass
seats-and-lock	leaving after lock (FR-RD-07.3): asks the rider to confirm, then keeps their locked share payable	Pass
seats-and-lock	leaving after lock (FR-RD-07.3): refuses a malformed confirmation flag	Pass
sos	raising an SOS: creates one alert per person, however often they tap, and tells nobody in the pool (FR-RD-15.2, FR-RD-15.4, FR-RO-13.2)	Pass
sos	raising an SOS: keeps the mobile number and a snapshot of the trip for the admins (FR-RD-15.2, FR-RD-02.7)	Pass
sos	raising an SOS: is only for people on the ride while the trip is under way (FR-RD-15.1)	Pass
sos	raising an SOS: lets only the person who raised it add a location or call it a false alarm (FR-RD-15.5)	Pass
sos	admins handling an SOS (CS455-65): shows the alert until acknowledged, then closes it with a note, auditing each step	Pass
sos	admins handling an SOS (CS455-65): keeps students out of the incident routes	Pass
sos	admins handling an SOS (CS455-65): keeps the original alert unchangeable in the database	Pass
trip-flow	starting pickup: waits for the lock, and only the owner can start it (FR-RO-09.1, NFR-RO-SEC-01)	Pass
trip-flow	starting pickup: cannot skip pickup, and cannot start once the departure window is over (FR-RO-09.1)	Pass
trip-flow	starting pickup: rejects an unknown step	Pass
trip-flow	during pickup: lets a rider check in and the owner mark them arrived or undo it (FR-RD-10.4, FR-RO-09.3)	Pass
trip-flow	during pickup: refuses a no-show until five minutes after the rider's pickup time (FR-RO-10.1, P-19)	Pass
trip-flow	during pickup: records a no-show after the grace period: off the ride, owes nothing, told how to dispute (FR-RO-10.2, FR-RO-10.3, FR-RD-10.3)	Pass
trip-flow	the trip and completion (FR-RO-09.4): stores the final pool and shares: the no-show's share is spread over the others	Pass
trip-flow	the trip and completion (FR-RO-09.4): counts unmarked riders as on board when the trip starts	Pass
trip-flow	the trip and completion (FR-RO-09.4): answers the live trip stamp to people on the ride only (FR-RD-10.1)	Pass